

⚖️ Failure Failure

picoCTF • General Skills

(Failover HAProxy — rate limit & bascule backup)

Flag : `picoCTF{f41l0v3r_f0r_7h3_w1n_ec6ea57b}`

🔍 Résumé de la faille

Un service web est réparti derrière un load balancer **HAProxy** avec deux serveurs backend : un serveur principal et un serveur de **backup**, qui n'est activé que si le principal est détecté "en panne". Le serveur backup a une variable d'environnement `IS_BACKUP=yes` qui affiche le flag. En **surchargeant volontairement** le serveur principal de requêtes (au-delà de sa limite de débit), on le fait paraître en panne aux yeux d'HAProxy, ce qui force le basculement automatique vers le serveur backup — révélant le flag.

Concept : exploiter un mécanisme de **haute disponibilité légitime** (failover) comme vecteur d'accès à un contenu normalement caché.

🔧 Méthodo — les étapes

1. Reconnaissance initiale

```
nmap -p- 10.129.172.194 # (exemple générique de reco)
```

Page d'accueil "Expense Tracker" affichant "No flag in this service". `gobuster` sur les dossiers/fichiers classiques ne donne rien — la faille n'est pas dans un contenu caché mais dans l'**architecture** du service.

2. Obtenir les fichiers de configuration (fournis par le challenge)

Config HAProxy (`haproxy.cfg`) :

```
frontend http-in
  bind *:80
  default_backend servers
backend servers
  option httpchk GET /
  http-check expect status 200
  server s1 *:8000 check inter 2s fall 2 rise 3
  server s2 *:9000 check backup inter 2s fall 2 rise 3
```

Code Flask (Python) :

```
limiter = Limiter(
    key_func=global_rate_limit_key,
    app=app,
    default_limits=["300 per minute"]
)

@app.errorhandler(429)
def ratelimit_exceeded(e):
    return "Service Unavailable: Rate limit exceeded", 503

@app.route('/')
def home():
    if os.getenv("IS_BACKUP") == "yes":
        flag = os.getenv("FLAG")
    else:
        flag = "No flag in this service"
    return render_template("index.html", flag=flag)
```

3. Analyser la logique — reconstituer la chaîne de cause à effet

- `s2` a le mot-clé `backup` dans sa ligne → ce serveur n'est actif QUE si `s1` est considéré en panne par HAProxy.
- `option httpchk GET /` + `http-check expect status 200` → HAProxy vérifie la santé de `s1` en attendant un code **200** sur `GET /`, toutes les 2 secondes (`inter 2s`).
- `fall 2` → après **2 échecs consécutifs** du health check, HAProxy marque le serveur comme down.
- Côté Flask : au-delà de **300 requêtes/minute**, le rate limiter renvoie un code **503** au lieu de 200.
- Sur `s2` (le backup), la variable d'environnement `IS_BACKUP=yes` est définie → le flag s'affiche uniquement sur ce serveur.

Le raisonnement complet :

Si j'envoie plus de 300 requêtes/minute à `s1`, il commence à répondre 503 au lieu de 200 → le health check HAProxy échoue 2 fois de suite → `s1` est marqué down → HAProxy bascule vers `s2` (backup) → `s2` a `IS_BACKUP=yes` → le flag s'affiche.

4. Exploiter : surcharger le service pour déclencher le failover

Une simple boucle `curl` séquentielle est trop lente (les requêtes s'enchaînent une par une, pas assez rapide pour dépasser 300/min). Il faut un outil de test de charge, capable d'envoyer beaucoup de requêtes **en parallèle**.

```
ab -n 500 -c 50 http://mysterious-sea.picoctf.net:PORT/
```

Résultat : 500 requêtes envoyées en 2.8 secondes, dont 226 en échec (`Non-2xx responses: 226`) — largement de quoi dépasser le seuil et faire échouer 2 health checks consécutifs.

5. Recharger la page

Une fois le failover déclenché, recharger `http://.../` affiche désormais le flag (HAProxy route maintenant vers `s2`) :

```
picoCTF{f41l0v3r_f0r_7h3_w1n_ec6ea57b}
```

🔧 Outils utilisés

Outil	Rôle
Lecture de config HAProxy	Comprendre l'architecture (frontend/backend, backup, health check)
Lecture de code Flask/Python	Comprendre la logique métier (rate limiting, condition d'affichage du flag)
<code>ab</code> (Apache Bench)	Générer une charge de requêtes HTTP en parallèle, pour déclencher le failover

✔ Réflexes à retenir

- Un load balancer avec un serveur `backup` ne devient actif que si le serveur principal échoue à son **health check** — comprendre les critères exacts du check (`httpchk` , `expect status` , `fall N`) révèle souvent comment forcer artificiellement ce basculement.
- Un **rate limiter mal exposé** (qui renvoie un code d'erreur visible par le health check du load balancer, comme ici 503) peut devenir un vecteur d'attaque : le mécanisme de protection anti-abus devient l'outil pour déclencher un comportement non voulu.
- Une boucle bash séquentielle (`for ... curl`) n'est pas toujours assez rapide pour du test de charge — utiliser un outil dédié (`ab` , `hey` , `wrk`) qui envoie des requêtes en **parallèle**.
- Toujours lire les fichiers de configuration fournis avec un challenge (HAProxy, nginx, docker-compose...) : ils révèlent souvent l'architecture complète et les conditions exactes à exploiter, sans avoir à deviner.

🏠 Dans la vraie vie

Ce challenge illustre un vrai type de vulnérabilité : un mécanisme de **haute disponibilité** (failover automatique) mal isolé peut devenir un vecteur d'attaque si le critère de bascule est manipulable depuis l'extérieur (ici, un simple pic de trafic contrôlé). Dans un vrai audit d'infrastructure, on vérifie que : les health checks ne sont pas facilement falsifiables, les serveurs "backup" n'exposent pas plus d'informations/de privilèges que le serveur principal, et le rate limiting est configuré pour ne pas impacter la disponibilité perçue par un load balancer en amont.